Introdução à Orientação a Objetos

Aula 04 — Construtores e sobrecarga

Programação Orientada a Objetos

Semana 4

Sumário

1. Introdução
2. Entendendo a construção de um objeto
3. Dependências obrigatórias
4. Construtor com parâmetros
5. Sobrecarga de construtores
6. Sobrecarga de métodos
7. Inicializando atributos
8. Delegando para outros construtores
9. Exercício de fixação
10. Encerramento

Revisão da aula anterior

- Atributo exposto = regra de negócio que pode ser burlada
- **Todo atributo de classe deve ser** `private`
- Encapsular = esconder os dados **e** esconder o *como*
- Getters e setters: acesso controlado, criados só quando necessários
- Nosso projeto: `Livro`, `Autor` e `CadastroDeLivros` já encapsulados

Mas ainda existe um buraco

```
1 Livro livro = new Livro();
2 livro.mostrarDetalhes();
```

- Esse código **compila** e **executa**
- Um livro sem nome, sem valor e sem autor existe em memória
- Fechamos o acesso aos atributos, mas não garantimos o **estado inicial** do objeto

Hoje

- Como um objeto é construído: o **construtor**
- Construtores com parâmetros: obrigando dependências
- **Sobrecarga** de construtores e de métodos
- Inicialização de atributos e delegação com `this()`

O que acontece no new?

```
1 Livro livro = new Livro();
```

- new pede à JVM que a classe Livro seja localizada e que seu **construtor** seja invocado
- O construtor da nossa classe se parece com:

```
1 public Livro() {
2 }
```

- Mas nós nunca escrevemos isso. Quem escreveu?

Construtor não é método

```java
public class Livro {
    public Livro() {   // construtor
    }

    public void mostrarDetalhes() {   // metodo
    }
}
```

	Construtor	**Método**
Nome	igual ao da classe	qualquer nome válido
Retorno	não declara retorno	declara (`void` inclusive)
Chamada	só com `new`	pelo objeto, quantas vezes quiser

O compilador escreve por você

Sempre que a classe não tiver **nenhum** construtor declarado, o compilador cria um vazio.

Quer a prova? Use o `javap`, que já vem no JDK, sobre a classe compilada:

```
> javap Livro
Compiled from "Livro.java"
public class Livro {
  public Livro();
  public void mostrarDetalhes();
  public boolean aplicaDescontoDe(double);
}
```

O construtor `public Livro();` está lá, mesmo sem nunca termos digitado.

Todo código do construtor roda a cada new

```java
public class Livro {
    private String nome;
    // demais atributos

    public Livro() {
        System.out.println("novo livro criado");
    }
}
```

```java
Livro livro1 = new Livro();
Livro livro2 = new Livro();
Livro livro3 = new Livro();
```

Saída: `novo livro criado` (três vezes)

Regra importante

O compilador só cria o construtor vazio quando a classe **não tem nenhum** construtor declarado.

- Declarou um construtor? O compilador para de criar o dele
- Guarde essa regra — ela vai quebrar nosso código daqui a poucos slides

Nova regra de negócio

Todo `Livro` precisa ter um `Autor`.

- Não é um dado opcional: é uma **dependência** do livro
- Como o nosso código garante isso hoje?

Como está hoje

```java
Autor autor = new Autor();
autor.setNome("Rodrigo Turini");

Livro livro = new Livro();
livro.setNome("Java 8 Pratico");
livro.setValor(59.90);
livro.setAutor(autor);
```

- Funciona — mas por alguns instantes o `Livro` existe em memória **sem autor**
- E se alguém esquecer a linha do `setAutor`?

O que acontece se esquecermos?

```java
Livro livro = new Livro();
livro.setNome("Java 8 Pratico");
livro.setValor(59.90);
// esqueceram o setAutor...

livro.mostrarDetalhes();
```

Resposta: **nada**.

- Compila, executa e cadastra um livro sem autor
- Ninguém obriga o programador a chamar o `setAutor`

Construtores podem receber parâmetros

```java
public class Livro {
    private String nome;
    private double valor;
    private Autor autor;

    public Livro(Autor autor) {
        this.autor = autor;
    }
}
```

- O construtor assumiu a responsabilidade do `setAutor`
- `this.autor` é o atributo; `autor` sozinho é o parâmetro
- Agora o autor é exigido **no momento da criação** do objeto

E o código antigo para de compilar

```
1 Livro livro = new Livro();
2 // erro de compilacao:
3 // constructor Livro in class Livro cannot be applied to given types;
4 //   required: Autor
5 //   found:    no arguments
```

- Como declaramos um construtor, o compilador não cria mais o vazio
- Só existe **um** jeito de criar um livro: passando um autor
- Mais uma vez: erro de compilação aqui é uma **vitória**

Criando o livro do jeito certo

```java
Autor autor = new Autor();
autor.setNome("Rodrigo Turini");
autor.setEmail("rodrigo.turini@caelum.com.br");

Livro livro = new Livro(autor);
livro.setNome("Java 8 Pratico");
livro.setValor(59.90);
```

- Impossível existir um `Livro` sem `Autor` — o compilador garante
- A regra de negócio virou uma restrição da linguagem

O CadastroDeLivros adaptado

```java
public class CadastroDeLivros {
    public static void main(String[] args) {
        Autor autor = new Autor();
        autor.setNome("Rodrigo Turini");
        autor.setEmail("rodrigo.turini@caelum.com.br");
        autor.setCpf("123.456.789.10");

        Livro livro = new Livro(autor);
        livro.setNome("Java 8 Pratico");
        livro.setDescricao("Novos recursos da linguagem");
        livro.setValor(59.90);
        livro.setIsbn("978-85-66250-46-6");

        livro.mostrarDetalhes();
    }
}
```

Mantemos o setAutor?

```java
public void setAutor(Autor autor) {
    this.autor = autor;
}
```

- Depende da necessidade: faz sentido **trocar** o autor de um livro já cadastrado?
- Se a livraria precisar corrigir o autor depois, mantenha
- Se não precisar, apague — setter só existe quando é realmente necessário
- Lembre-se da aula passada: cada setter é uma porta a mais na classe

E se quisermos as duas formas?

```java
public class Livro {
    private String nome;
    private Autor autor;

    public Livro(Autor autor) {
        this.autor = autor;
    }

    public Livro() {
    }
}
```

- Uma classe pode ter **mais de um** construtor
- Isso se chama **sobrecarga** (*overload*) de construtor

A regra da sobrecarga

Os construtores precisam ter **listas de parâmetros diferentes**:

```
1 public Livro() { }
2 public Livro(Autor autor) { }
3 public Livro(Autor autor, String nome) { }
```

Isso **não** compila:

```
1 public Livro() { }
2 public Livro() {
3     System.out.println("novo livro criado");
4 }
```

Quando alguém escrever `new Livro()`, qual dos dois seria chamado?

Decisão de projeto

- Poder criar dois construtores não significa que **deva**
- Nossa regra de negócio diz: todo livro tem autor
- Se mantivermos o `Livro()` vazio, a regra volta a ser furada

Na nossa livraria fica apenas o construtor `Livro(Autor autor)`.

Métodos também podem ser sobrecarregados

- Dois ou mais métodos da mesma classe podem ter o **mesmo nome**
- Contanto que as declarações de parâmetros sejam **diferentes**
- O compilador escolhe a versão cujos parâmetros combinam com os argumentos da chamada

Um mesmo nome representa a **ação geral**; cada versão trata de um caso específico.

Na nossa livraria

```java
public boolean aplicaDescontoDe(double porcentagem) {
    if (porcentagem > 0.3) {
        return false;
    }
    this.valor -= this.valor * porcentagem;
    return true;
}

public boolean aplicaDescontoDe(double porcentagem, String motivo) {
    System.out.println("Desconto aplicado por: " + motivo);
    return this.aplicaDescontoDe(porcentagem);
}
```

A segunda versão **reaproveita** a primeira em vez de repetir a regra.

Chamando as duas versões

```
1 livro.aplicaDescontoDe(0.1);
2 livro.aplicaDescontoDe(0.1, "promocao de lancamento");
```

- Quem usa a classe escolhe a versão mais conveniente
- Não precisa decorar dois nomes diferentes para a mesma ação

Assinatura de um método

assinatura = **nome** + **lista de parâmetros**

- O tipo de retorno **não** faz parte da assinatura
- Dois métodos da mesma classe não podem ter a mesma assinatura

```
1 public void aplicaDescontoDe(double porcentagem) { }
2
3 // Erro! mesma assinatura, so muda o retorno
4 public boolean aplicaDescontoDe(double porcentagem) { }
```

Sobrecarga e conversão automática de tipos

```java
public void mostra(int x) {
    System.out.println("Dentro de mostra(int): " + x);
}

public void mostra(double x) {
    System.out.println("Dentro de mostra(double): " + x);
}
```

```java
byte b = 99;
objeto.mostra(b);       // chama mostra(int) - conversao automatica
objeto.mostra(10);      // chama mostra(int)
objeto.mostra(10.1);    // chama mostra(double)
```

A conversão só entra em cena quando **não existe** uma versão que combine exatamente.

Você já usa sobrecarga há semanas

```java
System.out.println("Java 8 Pratico");    // println(String)
System.out.println(59.90);               // println(double)
System.out.println(10);                  // println(int)

Math.abs(-5);       // abs(int)
Math.abs(-5.9);     // abs(double)
```

- Em linguagens sem sobrecarga, cada versão precisaria de um nome diferente
- Você precisa lembrar de **um** nome; o compilador escolhe a versão certa

Cuidado com o exagero

- Nada impede sobrecarregar métodos que não têm relação entre si
- Mas sobrecarga **sugere** parentesco entre as operações
- Exemplo ruim: um `calcula(int)` que eleva ao quadrado e um `calcula(double)` que tira a raiz

Sobrecarregue apenas operações **intimamente relacionadas**.

O problema do ISBN

O número de ISBN pode demorar dias para ficar pronto. E se o livro for cadastrado antes?

```
Mostrando detalhes do livro
Nome: Java 8 Pratico
Descricao: Novos recursos da linguagem
Valor: 59.9
ISBN: null
```

- `null` significa: a referência não aponta para nenhum objeto
- Diferente dos tipos primitivos, objetos como `String` não têm valor padrão

Valores *default* dos tipos primitivos

Tipo	Valor padrão
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`char`	`'\u0000'`
`boolean`	`false`
qualquer referência (`String`, `Autor`, ...)	`null`

- Vale para atributos de classe — variáveis locais precisam ser inicializadas

Inicializando no construtor

```java
public Livro(Autor autor) {
    this.autor = autor;
    this.isbn = "000-00-00000-00-0";
}
```

Agora, um livro sem ISBN cadastrado imprime:

```
Nome: Java 8 Pratico
Valor: 59.9
ISBN: 000-00-00000-00-0
```

Inicializar atributos é um dos usos mais comuns do construtor.

Código repetido entre construtores

```java
public Livro(Autor autor) {
    this.autor = autor;
    this.isbn = "000-00-00000-00-0";
}

public Livro() {
}
```

- Quem criar o livro pelo construtor vazio fica **sem** o ISBN padrão
- Copiar a linha para os dois construtores resolveria — e criaria duplicação

A palavra reservada this()

```java
public Livro(Autor autor) {
    this();
    this.autor = autor;
}

public Livro() {
    this.isbn = "000-00-00000-00-0";
}
```

- `this()` chama **outro construtor da mesma classe**
- A inicialização comum fica em um lugar só
- A chamada a `this()` precisa ser a **primeira instrução** do construtor

this x this()

`this.autor`	referência ao **atributo** do objeto atual
`this()`	chamada a **outro construtor** da mesma classe

```java
public Livro(Autor autor) {
    this();              // chama o construtor Livro()
    this.autor = autor; // acessa o atributo autor
}
```

Mesma palavra, dois usos diferentes — repare nos parênteses.

Construtores na classe `Produto`

Partindo da `Produto` encapsulada na aula passada:

```
1 public class Produto {
2     private String nome;
3     private double preco;
4     private int quantidadeEmEstoque;
5
6     public boolean retiraDoEstoque(int quantidade) { ... }
7 }
```

1. Crie um construtor que exija `nome` e `preco`, e inicialize o estoque em `0`
2. Sobrecarregue com um segundo construtor que receba também a quantidade inicial, **delegando** com `this()`
3. Sobrecarregue `retiraDoEstoque()` sem parâmetros, retirando 1 unidade
4. Verifique: ainda é possível criar um `Produto` sem nome?

Uma solução possível

```java
public class Produto {
    private String nome;
    private double preco;
    private int quantidadeEmEstoque;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
        this.quantidadeEmEstoque = 0;
    }

    public Produto(String nome, double preco, int quantidadeEmEstoque) {
        this(nome, preco);
        this.quantidadeEmEstoque = quantidadeEmEstoque;
    }
}
```

A sobrecarga do método

```java
public boolean retiraDoEstoque(int quantidade) {
    if (quantidade > this.quantidadeEmEstoque) {
        return false;
    }
    this.quantidadeEmEstoque -= quantidade;
    return true;
}

public boolean retiraDoEstoque() {
    return this.retiraDoEstoque(1);
}
```

- A versão sem parâmetros **delega** para a versão completa
- A regra do estoque continua escrita em um lugar só

Resumo da aula

- Construtor: mesmo nome da classe, sem retorno, executado no `new`
- O compilador cria o construtor vazio — até você declarar o seu
- Construtor com parâmetro obriga dependências no momento da criação
- **Sobrecarga**: mesmo nome, listas de parâmetros diferentes
- Assinatura = nome + parâmetros (o retorno não conta)
- Construtor também serve para inicializar atributos
- `this()` delega para outro construtor — sempre na primeira linha

Próxima aula

- **Pacotes, constantes e membros estáticos**
- Nosso projeto vai deixar de ser um monte de arquivos soltos
- Vamos entender de vez os modificadores *default* e `protected`

Tarefa: lista de exercícios da aula 04.